

Fault Injection Attacks Against zkSTARKs

Alexander Dalton¹, Daniel Page² and Markus Schafnegg³

¹ Independent

² University of Bristol, UK

³ [[alloc] init], United States

Abstract. Fault injection attacks targeting schemes with zero-knowledge (ZK) properties are relatively absent in the wider literature. One of the few examples has recently shown a ZK signature scheme to be vulnerable to fault injection attacks. In this paper, we detail candidate fault injection attacks against zero-knowledge scalable transparent argument of knowledge (zkSTARK) provers, designed to violate zero knowledge capabilities. zkSTARK proving systems are complex, with a huge amount of diversity in implementation specifics. We match the variety within the STARK implementation ecosystem, proposing a variety of fault injection attacks against different popular algorithmic primitives. To the best of our knowledge, this marks the first exploration of the fault injection surface of zkSTARKs, and of the wider class of general-purpose ZK proving systems.

Keywords: Fault Injection · Zero Knowledge Proofs · zkSTARKs

1 Introduction

Zero-knowledge protocols (ZKPs) involve two parties, namely a *prover* $\mathcal{P}$ and a *verifier* $\mathcal{V}$, and allow $\mathcal{P}$ to convince $\mathcal{V}$ that a statement is valid (or true) without revealing any other information (i.e., beyond validity of the statement). A given ZKP can be classified as being either interactive or non-interactive, a non-interactive zero-knowledge (NIZK) protocol, with the latter often constructed from the former, e.g., via application of the Fiat-Shamir transformation [FS86]. Whereas an interactive ZKP allows $n \geq 1$ rounds of communication between $\mathcal{P}$ and $\mathcal{V}$, a NIZK protocol implies $n = 1$ round of communication, in which $\mathcal{P}$ convinces $\mathcal{V}$ that the statement is valid using a single communicated message.

One can view the step from a ZK to a NIZK protocol as optimising with respect to interactivity to enable practicality, with further sub-classes of ZKP then optimising for other properties. For example, a ZKP is termed a zero-knowledge succinct non-interactive argument of knowledge (zkSNARK) if it affords *non-interactivity*, and *succinctness*, the latter property requiring proofs are small. A ZKP is termed a zkSTARK if it additionally affords *transparency*, avoiding the trusted initialisation phase required by a zkSNARK, and *scalability*, allowing proof generation and verification to remain efficient even for complex statements. Note that the term *argument* rather than *proof* reflects assumption of a computationally bounded $\mathcal{P}$. Associated frameworks for their construction are instantiated using 1) a mechanism to represent the statement such as arithmetisation for circuit-like statements, and 2) a proof system, potentially translated (or compiled) into the resulting protocol. For example, use of a hash- rather than DLP-based proof system, as typical in a zkSTARK and a zkSNARK respectively, can afford plausible post-quantum security.

The Threat Landscape for ZK-Based Constructions. Note that the ZK property underlying ZKPs can be intuitively aligned with many use-cases, but particularly those which involve privacy preservation. One example is use-cases which depend on interaction

between an edge device and a blockchain, such as authorising the transfer of a private cryptocurrency [BSCG⁺14]. This and other examples can therefore pitch ZKPs within contexts for which the threat of implementation attacks is pertinent. For example, one can imagine an attacker targeting an implementation executing on some target device using passive, i.e., side-channel, and/or active, i.e., fault injection, techniques.

The framing of a ZKP as a complex, composite protocol rather than a primitive hints at various challenges with respect to mitigation of such attacks. First, the attack *surface* can extend beyond a single target device into third-party devices (and/or wider systems). Examples include those required to support trusted initialisation, and/or computational off-load (as may be required for proof generation from a constrained edge device). Note that the relationship between such devices can be non-trivial in terms of function, but also trust. Second, the attack *goal* can extend beyond key recovery. Although recovery of the (secret) proof generation key is the most obvious goal, any more general invalidation of the ZK property could also be deemed a valid attack. Third, the number and complexity of components which comprise a given ZKP often leads to a high-level implementation being compiled into a lower-level, deployed executable. In common with non-ZKP settings (see, e.g., [SCA18, SLP⁺25]), this fact leads to the potential for and therefore also the need to mitigate compiler-induced vulnerabilities.

Scope and Contributions. We suggest that, in part because of the relative immaturity of ZKPs in relevant contexts, there is a shortage of work on and thus understanding of implementation attacks and associated countermeasures. The most relevant related work is that of Mondal et al. [MAKK24], who focus on ZK-based post-quantum digital signature schemes. By injecting a single fault, they were able to invalidate the ZK property and allow $\mathcal{V}$ to learn information about secret data involved. Other, less directly relevant related work includes successful application of side-channel attacks. Examples include extraction of secret data in fast Reed–Solomon interactive oracle proof of proximity (FRI)-based ZK membership proofs [MRS25] and in ZK-based cryptocurrencies [TBP20].

The rapid evolution and deployment of ZKPs as a component of numerous, diverse use-cases implies that extension of work in this field is of some importance. The high-level goal of this paper is to do so, focused on study of fault injection as applied to zkSTARKs specifically. In more concrete terms, we make the following contributions:

1. In Section 3 we prove an intermediate lemma that equates learning elements of a private polynomial generated during the construction of a zkSTARK proof to learning elements of the underlying private transcript in zkSTARK protocols.
2. Following this intermediate result we develop and evaluate various attacks, differing with respect to the target operation or fault model. These include an attack against Horner-based polynomial evaluation based on a stuck-at-zero fault model, an attack against recursive and decimation-in-frequency (DIF) Cooley–Tukey NTT-based polynomial evaluation based on a loop-skip fault model, and an attack against decimation-in-time (DIT) Cooley–Tukey NTT-based polynomial evaluation based on a single-bit-upset fault model.
3. After each attack is detailed, it is verified experimentally in software, and, for the Cooley–Tukey attacks, also in instruction-level RISC-V simulation.
4. We further develop and discuss various countermeasures, with various trade-offs in terms of cost and capability.

Note that all associated material (e.g., source code relating to all software implementations) is available under an open source license.¹

¹URL redacted for submission, code submitted as supplementary material. All code will be open sourced prior to publication.

2 Preliminaries

2.1 Notation

Throughout the paper, we write $\mathcal{P}$ and $\mathcal{V}$ for the two parties of a proof system, namely the prover and the verifier. We work over a finite field $\mathbb{F}$, and we use the lowercase letters f , p , and c for polynomials over $\mathbb{F}$. The statement that $\mathcal{P}$ wants to prove concerns a private execution transcript A , which is the sequence of field elements produced by running the computation of interest, and whose length we denote by $|A|$. Two evaluation domains appear throughout our description. The first is a small trace domain $G \subset \mathbb{F}$ on which the transcript lives, and the second is a strictly larger extension domain $L \subseteq \mathbb{F}$ with $|L| = \rho^{-1} |A|$, where $\rho^{-1} > 1$ is a small blow-up factor. Finally, commitments are realised as Merkle trees, and we write M_f and M_p for the trees committing to the trace polynomial and to the composition polynomial, respectively. Requesting the value stored at a given leaf together with its authentication path is what we call opening the commitment. All logarithms are taken to base 2.

2.2 STARKs

STARKs are a class of proof systems, and a complete description is beyond the scope of this paper. We refer the reader to [BBHR18b] for the full construction. In this section we briefly describe the parts of the construction that are relevant for our attacks, and we omit the components that we do not make use of, such as the internals of the low-degree test. We focus on univariate STARKs, where the data of the prover is encoded into univariate polynomials. Our attacks may also apply in the multivariate setting, but we did not test this case.

At a high level, a STARK allows $\mathcal{P}$ to convince $\mathcal{V}$ that a given computation was carried out correctly, without $\mathcal{V}$ having to recompute it. If the zero-knowledge feature is used, this is done without revealing any private detail of the computation beyond the fact that it was performed correctly. In general, the construction can be split into the five steps described in the following, which we also summarise in Fig. 1. Our attacks target the second of these steps, namely the one where the transcript is encoded into a polynomial.

Arithmetisation. In the first step, the computation is written as an execution trace, which is a table of field elements. Each column of this table follows one register over time, and each row corresponds to one step of the computation. Reading a column from top to bottom gives the private transcript A . The correctness of the computation is then expressed by a set of public algebraic constraints that every honest trace must satisfy. Here, boundary constraints fix certain cells to fixed values (e.g., the inputs and outputs of the computation), while transition constraints relate each row to the next one and in this way encode the step function of the computation. A trace is valid if and only if all of these constraints vanish on every row.

Trace Polynomials. The trace domain G is chosen as a multiplicative subgroup of $\mathbb{F}$ of size $|A|$, generated by an element g , so that $G = \{g^0, g^1, \dots, g^{|A|-1}\}$. Each column of the trace is then interpolated into the unique polynomial $f : G \rightarrow \mathbb{F}$ of degree smaller than $|A|$ that agrees with the column on G , i.e., $A = \{f(x) \mid x \in G\}$. This step is essential for our attacks. Indeed, the interpolation defines a bijection between the $|A|$ coefficients of f and the $|A|$ entries of the transcript, which means that the coefficients of f are as sensitive as the transcript itself. In particular, anybody who learns these coefficients can reconstruct A , a fact that we make precise in Section 3. For this reason, the coefficients must never leave $\mathcal{P}$, and the evaluations $f(x)$ for $x \in G$ must never be opened to $\mathcal{V}$.

Low-Degree Extension and Commitment. In order to make the polynomial testable, $\mathcal{P}$ evaluates f on the larger extension domain L . The resulting vector of evaluations forms a Reed–Solomon codeword, and its redundancy is what the subsequent proximity test relies on. These evaluations are stored in the leaves of a Merkle tree M_f , whose root $\mathcal{P}$ sends to $\mathcal{V}$ as a binding commitment. From this point on, $\mathcal{V}$ can ask for any leaf to be opened, and the corresponding authentication path prevents $\mathcal{P}$ from changing a value afterwards. It may happen that $G \subset L$.² In this case, the evaluations on G are excluded from M_f , since opening one of them would directly reveal an entry of the private transcript to $\mathcal{V}$.

Composition Polynomial. Using random weights provided by $\mathcal{V}$, the individual constraints are then combined into a single composition polynomial p . This polynomial is constructed such that it is of low degree if and only if the trace satisfies all constraints, which is in turn equivalent to the constraint polynomial c vanishing on the entire domain G . The prover evaluates p over L and commits to it in a second Merkle tree M_p . In this way, checking that the computation was carried out correctly reduces to checking that p is of low degree. We note that only M_f is relevant for our attacks, and we mention M_p only for completeness.

Low-Degree Test and Query Phase. Finally, $\mathcal{P}$ and $\mathcal{V}$ run the FRI protocol [BBHR18a], an interactive oracle proof which shows that the committed evaluations are close to a low-degree codeword. The protocol proceeds in rounds, and each round halves the degree that is being tested. It ends with a query phase, in which $\mathcal{V}$ chooses a number of random positions in L , and $\mathcal{P}$ opens M_f , M_p , and the intermediate FRI commitments at these positions, together with the corresponding authentication paths. The number of queried positions is determined by the security parameter. If all openings are consistent and satisfy the FRI relation, then p is of low degree except with a small and fixed soundness error, and hence the constraints hold and the proof is accepted. As we will see, it is exactly these openings of M_f that our attacks try to steer towards values carrying entries of the transcript.

It is common to apply the Fiat–Shamir transform [FS86] in order to turn the interactive parts of the protocol into a non-interactive proof. In this case, all randomness of the verifier (i.e., the constraint weights, the FRI folding challenges, and the query positions) is instead derived by hashing the transcript of messages exchanged so far. As we discuss where relevant, this non-interactivity makes some of our attacks harder, since the query positions can no longer be chosen by the adversary. On the other hand, many STARK implementations can be configured to run interactively (see Table 1), which makes the corresponding attacks easier. We take a pessimistic approach in this work and assume the protocol is operating non-interactively unless the component we are attacking sees use exclusively in an interactive configuration (as is the case for Horner polynomial evaluation).

2.3 Zero Knowledge In STARKs

Zero knowledge in STARKs is derived from the fact that the verifier never learns any elements of the private transcript A , nor any coefficients of the secret polynomial f (which, given publicly known G , could be used to reconstruct A).

We note that in some cases, FRI and the entire proving stack is used without actually activating the zero-knowledge feature. This is the case for many popular zkVMs, where the main properties exploited are succinctness and computational integrity rather than enforcing privacy [RIS23, Suc].

However, there are use cases (e.g. [Azt21]) where ZK is indeed needed. In these scenarios there exist multiple ways of ensuring privacy. For example, when focusing on the

²Many protocols carefully choose g and L in order to avoid this case.

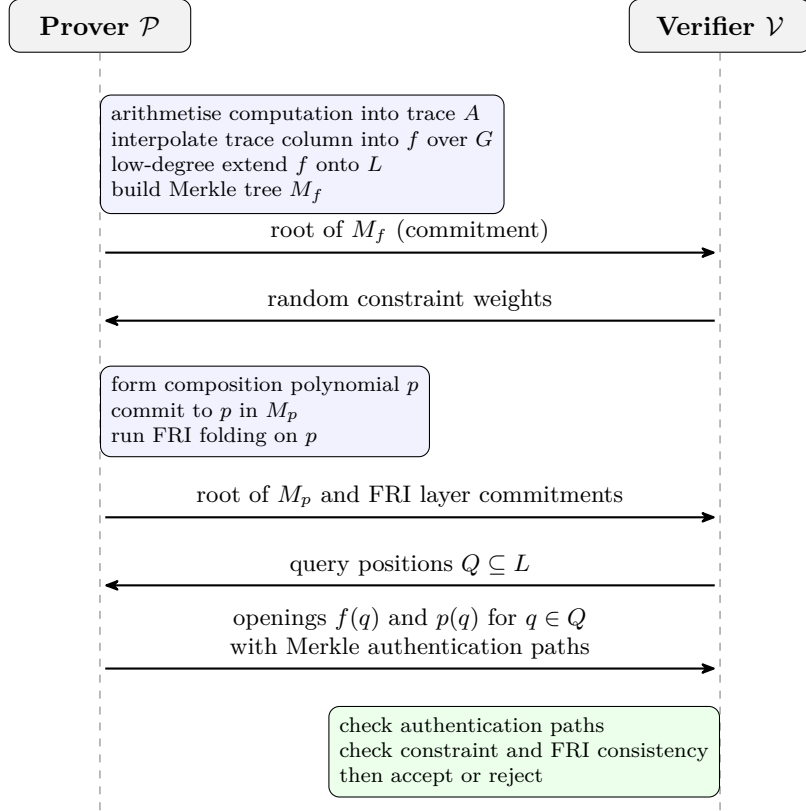


Figure 1: High-level message flow of a univariate STARK, with $\mathcal{P}$ on the left and $\mathcal{V}$ on the right. The boxes denote local computations, while the arrows denote exchanged messages. When the Fiat–Shamir transform is applied, $\mathcal{P}$ computes the messages from $\mathcal{V}$ as hashes of the previous transcript instead of receiving them. Our attacks target the interpolation step in the top box, and they do so by influencing which evaluations of f are eventually opened to $\mathcal{V}$.

trace in particular, one approach consists of adding random values to an existing witness column in the trace, in order to change the length of the trace to a more suitable number (for example, a power of 2) and to mask the potentially private witness values. We refer to [HK24] for details on this approach and an overview of different techniques.

An additional step consists of masking the Merkle tree leaves with random paddings, in order to prevent information leakage when opening values in the course of the protocol. This technique is described in more detail in [BCS16].

For our attacks, we have to consider a scenario where ZK is enabled, otherwise they would be meaningless. In particular, considering the trace column interpolations, the crucial part is that we still need to compute the coefficients of a polynomial, independently from the approaches sketched above. The coefficients are in general different when enabling zero knowledge with those approaches, but the resulting polynomial will still evaluate to the same witness value at a fixed chosen generator power. Hence, these countermeasures do not prevent the attacks described in this paper.

2.4 ZK Proof Attack Surface

When considering proving system security, a prover’s host system is an obvious target. However, the attack surface for proving system vulnerabilities goes beyond compromised

provers, and can be quite subtle. Many ZK systems have a complicated relationship between different components and third parties with different degrees of trust. A somewhat common complication involves delegating proof generation to a powerful trusted third party [Mid26]. Constructing zero knowledge proofs is computationally expensive, and as such, some existing projects suggest offloading proof generation to a trusted third party service. When proof generation is offloaded to a third party the user risks that even if a third party has honest intentions, they may be unknowingly compromised and subject to adversarial attack.

For example, the trusted third party may well be running a proof-generation-as-a-service, and running multiple workloads on shared hardware. This parallelisation risks honestly-run proof generation interacting unpredictably with external malicious workloads. Furthermore, under-provisioned hardware capacity can produce subtle and infrequent bugs which are exacerbated by workloads supplied by unknown and potentially untrusted external users [ZDCT13]. Trusted third parties, especially those servicing many users, make for a tempting single point of failure for an adversarial actor and increase the attack surface of an individual interacting with a ZK proof system.

2.5 Threat Model

The attacks presented in this work require the adversary has one of three possible capabilities: the ability to induce *stuck-at-zero* faults, *loop-skips*, or *single-bit-upsets*. All are widely understood to be plausible fault models [KSV13].

Stuck-At-Zero. Stuck-At-Zero faults provide the adversary with the ability to fix a register to zero with some reliable probability and can be achieved by voltage glitching or with the use of lasers. It is a well known and reasonable attack model.

Loop-Skip. Instruction skips, and thereby loop skips, are a class of induced faults where instructions are skipped over. In the case of loop skips this can be achieved by skipping the control flow instructions on the return edge of the loop, causing the loop to terminate early. Alternatively, faulting the instruction that controls entry into the loop can result in the loop being skipped entirely. Manufacturing instruction skips can be done with clock glitching and are known to be reproducible with high probability.

Single-Bit-Upsets. Single-Bit-Upsets are faults that induce a single bit-flip in a target register. These types of faults are often induced with lasers targeting physical locations on hardware.

2.6 Existing Implementations

The varied ecosystem of zkSTARK implementations have a similarly varied approach to the algorithmic primitives, in this subsection we canvas existing implementations and detail the specific algorithms we will later be attacking. The attacks in this work target two key primitives: Horner polynomial evaluation of f [Hor19], and the Cooley–Tukey NTT algorithm [CT65] for batch evaluations of f . We construct three different attacks against the three most popular variants of the Cooley–Tukey NTT algorithm (recursive, iterative DIF, and iterative DIT).

Table 1 details the use of the two primitives, and any variants, in available STARK implementations. This table demonstrates the huge variety in the implementation details of zkSTARK provers. The attacks demonstrated in this work target the most popular implementation choices.

Table 1: Algorithms used for polynomial evaluation in public STARK implementations.

Implementation	Horner	Cooley–Tukey Variant
Stone Prover [Stab]	Interactive	Mixed iterative DIT/DIF
Winterfell [Fac]	Configurable	Recursive DIT
Distaff [Guia]	Interactive	Recursive DIT
ethSTARK [Staa]	Interactive	Iterative DIT/DIF
Stwo [Stac]	No	None: circle-domain FFT
Plonky2 [Polb]	Interactive	Iterative DIT
Plonky3 [Plo]	Interactive	Iterative DIF/DIT
RISC Zero [RISa]	Interactive	Recursive DIT/DIF
zkSync Boojum [Mata]	Interactive	Iterative DIF
Polygon zkEVM [Pola]	Interactive	Iterative DIT
Lambdaworks [Lam]	Interactive	Iterative DIT
SP1 [Suc]		Delegates to Plonky3
OpenZKP [0xP]	Interactive	Recursive DIT
Hodor [Matb]	Interactive	Iterative and recursive DIT
Valida [Val]		Delegates to Plonky3
Miden air-script [Mid]		Delegates to Winterfell
libSTARK [Eli]	No	None: additive/subspace FFT
genSTARK [Guib]	Interactive	None: direct evaluation

To illuminate a specific example, Stone Prover [Stab] drives security on the StarkNet L2 blockchain [Sta24]. StarkNet uses the performance characteristics of STARKs to reduce fees when interacting with the Ethereum network, and the ZK properties to drive private transactions [GDD⁺26].

2.7 Algorithm Definitions

The attacks presented in this work target Horner polynomial evaluation, and three of the most popular Cooley–Tukey NTT variants (DIT recursive, DIF iterative in-place, and DIT iterative in-place). These algorithms are used to generate evaluations of the private trace polynomial f , which are exposed to the verifier as the opened leaves of the Merkle tree M_f . We present the algorithms in the same order the attacks are presented in this work.

2.7.1 Horner Polynomial Evaluation

The basis of Horner polynomial evaluation involves rewriting an input polynomial $p(x) = a_0 + a_1x + \dots + a_nx^n$ as $p(x) = a_0 + x(a_1 + x(\dots))$. By doing this, the evaluation only requires n additions and n multiplications. For most implementations detailed in Table 1 Horner is only used to calculate evaluations of the private polynomial f when the proving system is configured to operate interactively. Horner polynomial evaluation is specified in Algorithm 1.

Algorithm 1 Evaluate Polynomial Using Horner’s Method

Require: Coefficients $[a_0, a_1, \dots, a_n]$, evaluation point x

Ensure: Polynomial value $p(x)$

- 1: $r \leftarrow a_n$
 - 2: **for** $i = n - 1$ down to 0 **do**
 - 3: $r \leftarrow r \cdot x + a_i$
 - 4: **return** r
-

2.7.2 Cooley–Tukey NTT

It is a common convention to make use of the Cooley–Tukey NTT algorithm as a trick to calculate batch polynomial evaluations. This trick works as a STARK implementation does not care what the extended evaluation domain L actually contains, and the output of an NTT is structurally identical to repeated evaluations of a polynomial over a large domain $\omega \in W$. The Cooley–Tukey algorithm is one of the most popular NTT algorithms, and as such, many implementations in Table 1 use a variant of Cooley–Tukey for batch polynomial evaluations. The Cooley–Tukey algorithm in general operates via a divide-and-conquer approach. It reorganises a single NTT into 2 smaller NTTs that must later be mixed. The DIT or DIF distinction refers to the order in which the smaller NTTs are executed, with the former performing small NTTs first and the latter performing large NTTs first.

Recursive Cooley–Tukey NTT. Much like the iterative variant of Cooley–Tukey, the recursive variant can be either DIT or DIF. The former is specified here, but the distinction is unimportant for the attacks specified later. The details of the algorithm are specified in Algorithm 2. The DIF variant would do the mixing before the recursive calls to invert the order in which elements are mixed.

Algorithm 2 Cooley–Tukey NTT (DIT, recursive)

```

1: procedure NTT-RECURSIVE-DIT( $a, n, p, \omega$ )
2:   if  $n = 1$  then
3:     return  $a$ 
4:   for  $j = 0$  to  $n/2 - 1$  do
5:      $a_{\text{even}}[j] \leftarrow a[2j]$ 
6:      $a_{\text{odd}}[j] \leftarrow a[2j + 1]$ 
7:    $\omega' \leftarrow \omega^2 \bmod p$ 
8:    $E \leftarrow \text{NTT-RECURSIVE-DIT}(a_{\text{even}}, n/2, p, \omega')$ 
9:    $O \leftarrow \text{NTT-RECURSIVE-DIT}(a_{\text{odd}}, n/2, p, \omega')$ 
10:   $\omega_{\text{current}} \leftarrow 1$ 
11:  for  $j = 0$  to  $n/2 - 1$  do
12:     $t \leftarrow \omega_{\text{current}} \cdot O[j] \bmod p$ 
13:     $y[j] \leftarrow (E[j] + t) \bmod p$ 
14:     $y[j + n/2] \leftarrow (E[j] - t + p) \bmod p$ 
15:     $\omega_{\text{current}} \leftarrow \omega_{\text{current}} \cdot \omega \bmod p$ 
16:  return  $y$ 

```

DIF Cooley–Tukey NTT. The iterative Cooley–Tukey NTT algorithm performs a sequence of mixing butterfly steps, where each round features a different block size, where the DIF variant starts with large blocks first. The algorithm is specified in Algorithm 3.

Algorithm 3 Cooley–Tukey NTT (DIF, in-place, iterative)

```

1: procedure NTT-DIF( $a, n, p, \omega$ )
2:   for  $s = \log_2 n$  down to 1 do
3:      $m \leftarrow 2^s$ 
4:      $\omega_m \leftarrow \omega^{n/m} \bmod p$ 
5:     for  $k = 0$  to  $n - 1$  by  $m$  do
6:        $\omega_{\text{current}} \leftarrow 1$ 
7:       for  $j = 0$  to  $m/2 - 1$  do
8:          $u \leftarrow a[k + j]$ 
9:          $v \leftarrow a[k + j + m/2]$ 
10:         $a[k + j] \leftarrow (u + v) \bmod p$ 
11:         $d \leftarrow (u - v + p) \bmod p$ 
12:         $a[k + j + m/2] \leftarrow \omega_{\text{current}} \cdot d \bmod p$ 
13:         $\omega_{\text{current}} \leftarrow \omega_{\text{current}} \cdot \omega_m \bmod p$ 
14:   Bit-reverse reorder  $a$ 
15:   return  $a$ 

```

DIT Cooley–Tukey NTT. Much like the iterative DIF Cooley–Tukey NTT algorithm, the DIT version performs mixing butterfly steps across different block sizes. In this case the blocks start small and become larger. The change renders the attack later presented against the DIF variant ineffective, and as such requires a different strategy to attack. The details of the algorithm are specified in Algorithm 4.

Algorithm 4 Cooley–Tukey NTT (DIT, in-place, iterative)

```

1: procedure NTT-DIT( $a, n, p, \omega$ )
2:   Bit-reverse reorder  $a$ 
3:   for  $s = 1$  to  $\log_2 n$  do
4:      $m \leftarrow 2^s$ 
5:      $\omega_m \leftarrow \omega^{n/m} \bmod p$ 
6:     for  $k = 0$  to  $n - 1$  by  $m$  do
7:        $\omega_{\text{current}} \leftarrow 1$ 
8:       for  $j = 0$  to  $m/2 - 1$  do
9:          $t \leftarrow \omega_{\text{current}} \cdot a[k + j + m/2] \bmod p$ 
10:         $u \leftarrow a[k + j]$ 
11:         $a[k + j] \leftarrow (u + t) \bmod p$ 
12:         $a[k + j + m/2] \leftarrow (u - t + p) \bmod p$ 
13:         $\omega_{\text{current}} \leftarrow \omega_{\text{current}} \cdot \omega_m \bmod p$ 
14:   return  $a$ 

```

2.8 Fault Injection via spike-fi

spike-fi is a fault injection simulation framework based on the **spike** RISC-V ISA simulator [risb]. Simulating the attacks at the instruction level provides a much closer approximation of mounting a real attack as compared with coarser software simulation. In particular, it forces us to consider the effects of the compiler on the program. Instead of deciding certain code paths should be "skipped", we instead have to isolate specific control flow instructions as targets for fault injection. For example, when attempting a loop-skip one must decide whether to attack the entry conditions to the loop, or whether it would suffice to perform an instruction skip on the return edge of the loop, resulting in a partial loop skip in which the first iteration is guaranteed to run. Similar to the software simulation, **spike-fi** does not simulate fault-induced side-effects, for example faults that

missed the target instruction/register and affected other parts of the program, potentially inducing a crash.

3 Attacks

In this section we detail our attacks against zkSTARK provers. We start by describing the overall methodology, followed by the details of each attack in turn, including theoretical basis, experimental results, and suggested countermeasures.

3.1 Intermediary Result

Lemma 1. *Discovering even a single coefficient a_i in the trace polynomial f that is not fixed by the public statement is sufficient to violate the zero knowledge property of a zkSTARK.*

Proof. Consider an adversary $\mathcal{A}$ capable of extracting a random coefficient a_i of f (together with its index i) from a public protocol transcript T with some non-negligible probability, i.e.,

$$P(\mathcal{A}(T) = a_i) \geq \frac{1}{\text{poly}(\lambda)},$$

where a_i denotes the true i -th coefficient of f .

Given two private transcripts $A_0 \neq A_1$ with different underlying secret witness values but the same public constraints, consider adversary $\mathcal{B}$ capable of distinguishing between a public zkSTARK protocol transcript of A_0 and A_1 given in Fig. 2. If $\mathcal{A}$ can extract coefficients of f with non-negligible probability, then $\mathcal{B}$ can distinguish between A_0 and A_1 with good probability.

Since $\mathcal{B}$ generates A_0 and A_1 itself, it can choose them so that the interpolated polynomials f_0 and f_1 differ in a constant fraction of their coefficients. Each invocation of $\mathcal{A}$ returns a random coefficient together with its index i , so the loop in Fig. 2 simply resamples until it lands on an index where $f_0[i] \neq f_1[i]$, which happens after a constant expected number of iterations. At such an index the extracted value a_i is decisive, i.e., it can match at most one of $f_0[i]$ and $f_1[i]$.

Zero knowledge requires that proofs produced from two valid private transcripts for the same public statement be computationally indistinguishable. This follows because a zero-knowledge simulator is independent of the witness, so the proofs of A_0 and A_1 are each indistinguishable from the same simulated proof, and hence from each other. The distinguisher above contradicts this requirement.

Therefore, learning any coefficient a_i that is not fixed by the public statement is sufficient to break the zero-knowledge property. $\square$

Remark 1. Learning a function of A does not automatically violate zero knowledge as some parts of A may be directly derived from the public constraints.

3.2 Experimental Methodology

The attacks presented in this work have been validated with two methods. The first is a software simulation, with a simple zkSTARK prover and integrated software-level fault model facsimile. The second method is `spike-fi`, a RISC-V instruction set simulator with fault injection capabilities. `spike-fi` was only used for Cooley–Tukey as those attacks, unlike the others proposed in this section, attack primitives along the default non-interactive zkSTARK proving path for many of the real-world implementations given in Table 1.

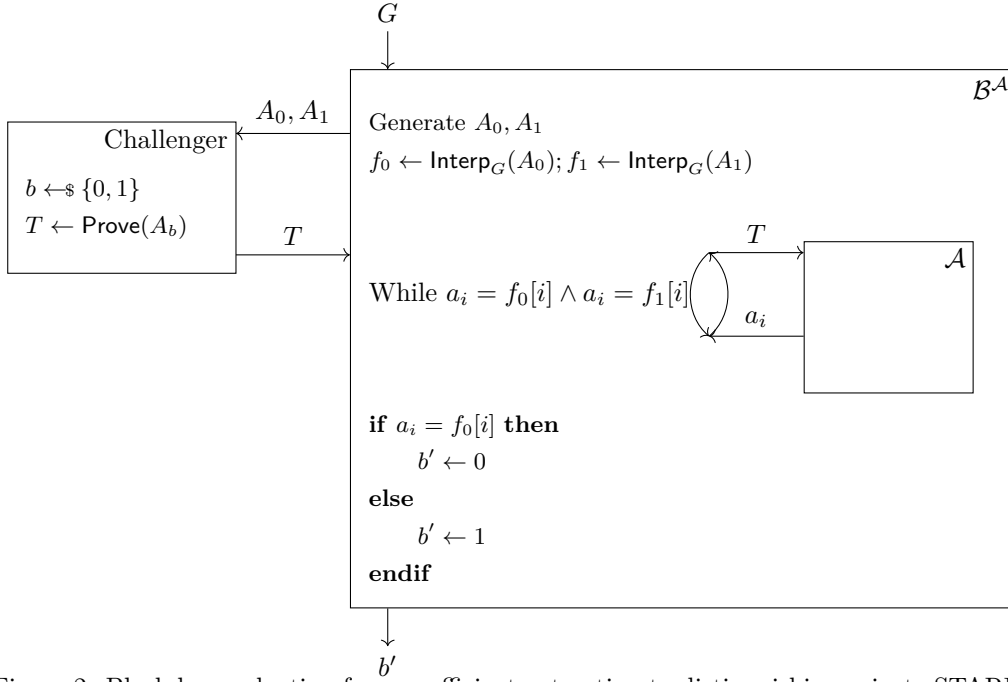


Figure 2: Black-box reduction from coefficient extraction to distinguishing private STARK execution transcripts from a public transcript. $\mathcal{B}$ wins if $b' = b$.

All experiments targeted proving over a simple private computation calculating the first n Fibonacci numbers, with o Merkle tree openings, and some probability of successfully inducing the target fault p_f . Success was determined by counting the number of secret values exposed in the public protocol transcript. If at least one secret value was exposed the attack was determined a success. In all cases we ran two sets of experiments, the first varying n and p_f and the second varying o and p_f . These two sets of experiments allowed us to gain insight into how the transcript size and security parameter affect the difficulty of attack, respectively. Note that the number of openings is linear in the number of bits of security set by the security parameter. More security requires more openings. When n varied, o was set to 64, which is in the approximate window for 128-bit security. When o varied, n was set to 512. In all cases the probability of successful fault injection varied between 0.0 and 1.0, with increments of 0.1, and each individual experimental configuration was repeated 30 times.

Software Simulation. The software simulation consisted of a zkSTARK prover written in Python, with additional control flow operations to approximate fault injection attempts. This facsimile does account for fault induced side-effects, such as corrupting the computation and/or causing a crash – either a fault was induced, or there was no effect. There are obvious criticisms to only verifying our attacks in simulation. However, simulation has been shown to be a powerful tool for efficiently demonstrating the efficacy of fault injection attacks [GS21].

spike-fi. The instruction-level simulation gives us a much more fine-grained simulation of the attack, forcing us to attack specific compilation artifacts rather than block of high level code. It is unable to completely simulate real-world fault injection side effects, such as misfiring faults, which may corrupt the computation.

Table 2: Horner stuck-at-zero attack successes in simulation, across 30 experiments, varying n and p_f , with $o = 64$.

$n \setminus p_f$	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
128	30	30	30	30	30	30	30	30	30	30
256	30	30	30	30	30	30	30	30	30	30
512	30	30	30	30	30	30	30	30	30	30
1024	30	30	30	30	30	30	30	30	30	30

Table 3: Horner stuck-at-zero attack successes in simulation, across 30 experiments, varying o and p_f , with $n = 512$.

$o \setminus p_f$	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
32	29	30	30	30	30	30	30	30	30	30
64	30	30	30	30	30	30	30	30	30	30
128	30	30	30	30	30	30	30	30	30	30
256	30	30	30	30	30	30	30	30	30	30

3.3 Attacking Horner Polynomial Evaluation

This is how some STARK implementations evaluate execution trace polynomial evaluations, generating M_f , particularly when operating interactively, see Table 1.

3.3.1 Strategy

Consider the algorithm presented in Algorithm 1. By resetting the cumulative total on line 3 we aim to set the output of the function to the value a_0 . Specifically, when the prover is calculating evaluations of f to construct M_f , immediately before the final iteration of the loop, we attack the register holding r , setting $r \leftarrow 0$. If successful, the entire evaluation has been corrupted and will simply return the lowest degree coefficient a_0 . The attack should be repeated for the generation of each leaf of the Merkle tree M_f . The verifier then discovers a_0 with high probability when shown openings of M_f .

The adversarial verifier can determine an attack to be successful if their proof verification fails. This would indicate one or more of the values opened are actually the degree zero coefficient of f . If two identical values are observed, these are very likely to be the value a_0 . Given Lemma 1, this is sufficient to circumvent zero knowledge.

3.3.2 Probability of Success

If we are able to set a given register to 0 with some probability p_f , then the probability of a faulted value in the o openings of M_f is given by

$$P(\text{success}) = 1 - (1 - p_f)^o.$$

3.3.3 Experimental Results

The experimental results given in Table 2 and Table 3 show that, across all experiments, in only a single run (at $p_f = 0.1$, $o = 32$) did the protocol transcript fail to include a private coefficient of f .

In order to validate the predicted probability of attack success, we conducted 4 experiments with a varied number of openings o sweeping across fault injection probabilities from 0.0 to 1.0. The results in Fig. 3 show the predicted results in a blue line and experimental results in orange dots. o increases with the security parameter. For realistically large o , the probability of success approaches 1.0 even for incredibly small fault injection probabilities.

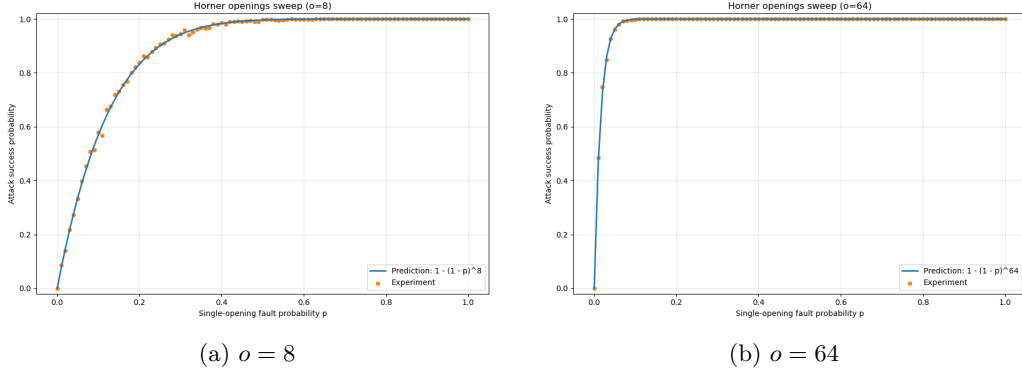


Figure 3: The attack success probability in simulation for the Horner stuck-at-zero attack for varied o . Predicted attack success given in a blue line.

3.3.4 Countermeasures

The Horner attack of Section 3.3 allows the outer loop to run to completion while clamping the accumulator register mid-iteration, so simple loop-bound checks do not defeat it. The most direct defense is to duplicate each Horner evaluation and abort on disagreement. As shown in Table 1, Horner is usually used only when the prover is configured to operate interactively, where it stands in for the Cooley–Tukey batch transform and is called once per element of the extension domain L . Duplicating it therefore doubles a phase of the interactive prover that is already substantial and the resulting performance hit is not negligible. We nonetheless recommend duplication as the default defense. The comparison itself is trivial. The two passes can be interleaved or run on distinct register files or code paths to harden against correlated faults, and the attacker is forced to mount a coordinated fault on both copies of a single call rather than a single-instruction glitch. Cheaper solutions of the kind we discuss for Cooley–Tukey in Section 3.4 below are not possible here, because a Horner attacker can target the specific evaluation whose leaf will later be revealed to the verifier. If an attack of this type is attempted, there is a high probability that output elements of the evaluation set are equal to the zero degree coefficient of f . An $\mathcal{O}(n)$ sweep across the output to check may be suitable.

3.4 Attacking Recursive Cooley–Tukey NTT

All attacks against Cooley–Tukey presented in this work try to skip the mixing steps to allow an adversary to extract coefficients of f from the corrupted output of the NTT algorithm, random elements of which are included as “openings” to the Merkle tree M_f as part of the public transcript. The recursive variant of Cooley–Tukey is given in Algorithm 2. In this case we target a recursive call such that many of the inputs only undergo a single round of mixing, and then attempt to correlate outputs and reverse the single round of mixing. We only need a single instruction skip in order for all even indexed values to undergo no more than a single round of mixing.

3.4.1 Strategy

In the recursive case the even and odd indexed input coefficients are split and form the input to two subsequent recursive calls, the results of which are then combined via butterfly operations. By attempting a single *instruction skip* on the first recursive call (on line 8, Algorithm 2), we aim to minimize the mixing done.

If the instruction skip is successful, the set of even coefficients only undergo a single

Table 4: Recursive Cooley–Tukey instruction skip attack successes in simulation, across 30 experiments, varying n and p_f , with $o = 64$.

$n \setminus p_f$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
128	0	5	2	7	21	20	18	23	23	28	30
256	0	5	7	8	13	13	14	18	24	24	25
512	0	1	3	8	10	10	7	15	13	18	14
1024	0	1	2	2	3	5	5	6	11	13	12

round of mixing (lines 11 through 15 in Algorithm 2). Given function $NTT : \mathbb{F}^n \rightarrow \mathbb{F}^n$ denoting the transform executed on input values to the target algorithm, at the end of execution, even values are unchanged $E \leftarrow E$, and odd values become $P \leftarrow NTT(O)$. The mixing steps then produce

$$\begin{aligned} y_j &\leftarrow E_j + \omega^j P_j, \\ y_{j+n/2} &\leftarrow E_j - \omega^j P_j. \end{aligned}$$

As such, assuming that at least two values spaced $n/2$ apart are observed as part of the openings of M_f , the adversary can extract a coefficient of f , a , by calculating

$$a = \frac{y_j + y_{j+n/2}}{2}.$$

Given Lemma 1, learning a coefficient of f is sufficient to subvert zero knowledge. Detection is possible by attempting to verify the proof. If the verification fails there is a good chance correlated inputs can be used to extract coefficients of f . Note that if the protocol is operating interactively, the attack can be made considerably easier by explicitly requesting openings of correlated values.

3.4.2 Probability of Success

The probability of success is given by the probability of successfully inducing a fault and the probability of witnessing two openings separated by $n/2$. Given a probability p_f of successfully inducing a fault, we have

$$P(\text{success}) = p_f \left(1 - \frac{2^o \binom{n/2}{o}}{\binom{n}{o}} \right).$$

3.4.3 Experimental Results

The attack is easier to mount for small input sizes n and many openings o . This is validated in the software simulation given in Table 4 and Table 5, and corroborated in the instruction-level `spike-fi` simulation given in Table 6 and Table 7. Smaller n and larger o maximise the probability that correlated faulted values are included in the public transcript.

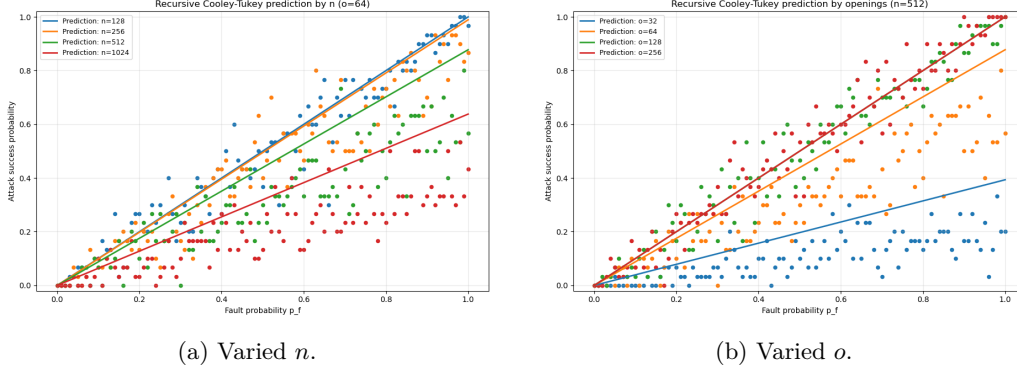
Fig. 4 shows the linear relationship between the probability of injecting the fault and the probability the attack is successful, adjusted as n and o are varied.

3.5 Attacking DIF Cooley–Tukey NTT

When switching to the iterative variants of Cooley–Tukey, skipping the mixing steps becomes more complex. Both DIF and DIT iterative Cooley–Tukey algorithms split the

Table 5: Recursive Cooley–Tukey instruction skip attack successes in simulation, across 30 experiments, varying o and p_f , with $n = 512$.

$o \setminus p_f$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
32	0	0	2	1	5	2	4	6	8	6	11
64	0	1	1	6	7	9	15	14	19	17	15
128	0	3	8	7	17	21	16	18	18	30	30
256	0	3	4	5	14	13	20	22	23	25	30

Figure 4: The attack success probability in simulation with varied n and o .Table 6: Recursive Cooley–Tukey instruction skip attack successes in **spike-fi**, across 30 experiments, varying n and p_f , with $o = 64$.

$n \setminus p_f$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
128	0	3	8	8	13	9	18	19	23	30	30
256	0	2	3	9	11	15	18	13	25	28	24
512	0	1	3	7	11	18	14	11	18	16	21
1024	0	2	0	3	7	7	12	10	10	10	12

input coefficients into blocks, and then mix the contents of those blocks. The difference between the two iterative variants is that DIF starts with large blocks that become smaller, and DIT starts with small blocks that become larger.

3.5.1 Strategy

In order to skip the mixing steps we target the return edge of the inner-most loop, which runs from lines 7 to 13 in Algorithm 3, with an *instruction skip*. If successful, each block only undergoes one step of mixing, preserving elements at the end of the block. Unfortunately, the final round of mixing is impossible to skip this way, as the blocks are of size 2, and as such only undergo a single mixing step. However, much in the same manner we extracted partially-mixed values in the recursive variant, if we see correlated values we can extract the secret input values.

Block sizes descend in powers of 2 from the full input size down to 2. Assuming the fault is injected successfully each time it is attempted, only the first and $m/2$ th elements in each block are mixed (where m is the current block size). As such, before the final round of mixing, only the evenly numbered elements will have undergone any mixing at all. The final step mixes neighbouring elements. This means, in much the same way we attacked recursive Cooley–Tukey, we can extract a pre-final-mix odd coefficient if we see the two results of a final mixing step.

Table 7: Recursive Cooley–Tukey instruction skip attack successes in **spike-fi**, across 30 experiments, varying o and p_f , with $n = 512$.

$o \setminus p_f$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
32	0	1	3	1	4	4	5	6	5	9	8
64	0	1	6	9	9	11	14	14	18	20	18
128	0	4	6	7	11	15	19	24	22	28	30
256	0	1	7	10	13	13	12	21	26	29	30

But first the elements are bit-reversed. Therefore, correlated items are once again separated by $n/2$ elements. Therefore, if enough instruction skips are successful and if the adversary sees two elements with indexes separated by $n/2$, the adversary can extract a secret coefficient of f via

$$a = \frac{y_j - y_{j+n/2}}{2}.$$

Note that this differs from the recursive case because odd indexed elements are unmixed instead of even indexed elements. Lemma 1 shows that this suffices to violate zero knowledge. Once again, a successful attack can be detected by attempting to verify the proof, which will fail if the fault is successfully induced.

Note that, as in the recursive case, if the protocol is operating interactively, the attack can be made considerably easier by explicitly requesting openings of correlated values.

3.5.2 Probability of Success

This attack is more complex to reason about than the recursive case. The more mixing steps that are skipped, the more likely we are to see corrupted correlated values in the public transcript.

Given a probability p_f of successfully inducing a fault, the probability of any one odd indexed element going into the final butterfly step unmixed is given by

$$P(\text{a given odd element is unmixed}) = p_f^{\log_2(n)-1}.$$

Therefore, the expected number of elements unmixed before the final butterfly is given by

$$\mathbb{E}[\text{unmixed elements}] = \frac{n}{2} p_f^{\log_2(n)-1}.$$

As such, with o elements appearing in the final transcript, the probability of successfully mounting the attack is given by

$$P(\text{success}) = 1 - \frac{\sum_{k=0}^{\min(\mu, o)} \binom{\mu}{k} 2^k \binom{n-2\mu}{o-k}}{\binom{n}{o}}, \quad \mu = \mathbb{E}[\text{number of unmixed elements}]$$

This is a pessimistic over-simplification and does not account for the ability to “retry” skipping the return loop edge if the previous attempts failed.

3.5.3 Experimental Results

Much like in the recursive case, our attack is easier for smaller values of n and larger values of o . This maximises the opportunity for correlated faulted results to appear in the public

Table 8: DIF Cooley–Tukey loop-skip attack successes in simulation, across 30 experiments, varying n and p_f , with $o = 64$.

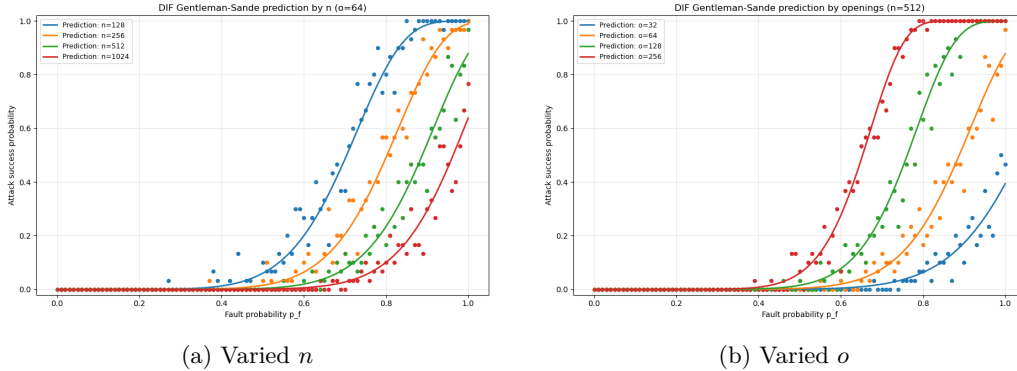
$n \setminus p_f$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
128	0	5	7	12	16	14	19	23	25	25	30
256	0	3	4	10	8	11	23	19	24	27	29
512	0	4	6	9	13	19	17	17	21	23	26
1024	0	3	2	4	12	14	11	11	10	24	21

Table 9: DIF Cooley–Tukey loop-skip attack successes in simulation, across 30 experiments, varying o and p_f , with $n = 512$.

$o \setminus p_f$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
32	0	3	1	4	3	8	11	3	17	9	13
64	0	3	7	9	10	13	19	16	20	21	29
128	0	2	5	10	11	16	16	20	24	29	30
256	0	5	4	10	11	19	19	20	24	29	30

transcript. This is reflected in the software simulation results in Table 8 and Table 9, and the instruction-level simulation in Table 10 and Table 11.

The relationship between the probability of success, n , and o is explored in the graphs presented in Fig. 5. These relationships are no longer linear, and instead show a sharp rise in probability of attack success when a fault induction probability threshold is reached.

Figure 5: The attack success probability in simulation with varied n and o . Predicted results are included as lines.

3.6 Attacking DIT Cooley–Tukey NTT

Unfortunately, as compared with DIF, the DIT variant of Cooley–Tukey inverts the order in which mixing blocks are sized. Instead of largest to smallest, the algorithm presented in Algorithm 4 mixes small blocks then larger blocks. This means the mixing stage which cannot be skipped by skipping the return edge of the loop happens first, not last. Subsequently, half of the initially mixed values undergo further mixing stages (the unskippable first iteration on subsequent loops), and as such reversing the initial mixing becomes intractable. The attack in which we skip the return edge of the loop has to be discarded entirely, and instead the entry conditions to the loop have to be attacked. Note that the approach detailed here also works for DIF Cooley–Tukey, and an attacker may wish to use this method, should it better align with their capabilities.

Table 10: DIF Cooley–Tukey loop-skip attack successes in **spike-fi**, across 30 experiments, varying n and p_f , with $o = 64$.

$n \setminus p_f$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
128	0	1	0	5	10	14	22	26	28	30	28
256	0	0	1	5	3	12	18	21	28	20	28
512	0	0	2	3	6	9	12	13	16	17	19
1024	0	0	1	0	1	6	7	9	7	12	15

Table 11: DIF Cooley–Tukey loop-skip attack successes in **spike-fi**, across 30 experiments, varying o and p_f , with $n = 512$.

$o \setminus p_f$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
32	0	0	1	0	3	1	3	6	2	5	3
64	0	0	1	4	7	10	12	17	13	15	18
128	0	0	3	10	11	22	22	30	27	30	30
256	0	2	11	21	29	30	30	30	30	30	30

3.6.1 Strategy

In much the same manner as the previous attack on Cooley–Tukey, we attempt to skip mixing stages such that the prover erroneously includes secret coefficients of f in the public protocol transcript. This is achieved by attempting to fix the control variable j on line 8 in Algorithm 4 to some large value (e.g. `0x80000000`, which is a single bit flip from the starting value of `0x00000000`). If this is successful, the entire mixing stage is skipped, resulting in all input coefficients being included in the output. If the attack is only partially successful (i.e. some loops are skipped but others only partially or not at all), only some of the secret input values will be unmixed. Note that, as in the recursive and DIF cases, if the protocol is operating interactively, the attack can be made considerably easier by explicitly requesting openings of correlated values. Once again, an induced fault can be checked by attempting to verify the proof, and a failure would indicate a successfully induced fault.

3.6.2 Probability of Success

Given a probability p_f of successfully inducing a fault, the probability that any one input value undergoes no mixing stages is given by

$$P(\text{a given input value is unmixed}) = p_f^{\log n}.$$

The probability of the attack succeeding is the inverse of the probability that no unmixed values appear in the o openings. As such, the probability that the attack succeeds is given by

$$P(\text{success}) = 1 - (1 - p_f^{\log n})^o.$$

3.6.3 Experimental Results

As with the previous attacks, small n and large o result in a higher likelihood of observing correlated values in the public transcript. The software simulation results in Table 12 and Table 13, and the instruction-level simulation in Table 15 and Table 14.

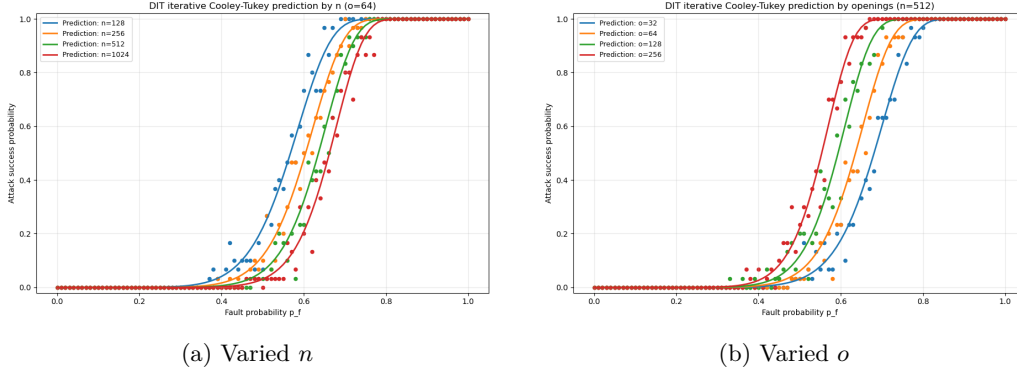
Much like with the previous Cooley–Tukey attacks we see a sharp increase in the attack success probability when the probability of inducing a fault reaches a threshold.

Table 12: DIT Cooley–Tukey loop-skip attack successes in simulation, across 30 experiments, varying n and p_f , with $o = 64$.

$n \setminus p_f$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
128	0	0	0	2	3	10	17	24	28	29	30
256	0	0	0	1	1	6	10	29	30	28	30
512	0	0	0	0	0	4	13	22	27	29	30
1024	0	0	0	0	0	3	4	16	25	30	30

Table 13: DIT Cooley–Tukey loop-skip attack successes in simulation, across 30 experiments, varying o and p_f , with $n = 512$.

$o \setminus p_f$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
32	0	0	0	0	0	0	12	22	27	28	30
64	0	0	0	0	0	4	7	24	27	30	30
128	0	0	0	0	1	6	12	24	29	30	30
256	0	0	0	0	3	6	11	25	30	30	30

Figure 6: The attack success probability in simulation with varied n and o .

3.7 Cooley–Tukey Attack Countermeasures

The Cooley–Tukey attacks depends on skipping the butterfly mixing steps while leaving the surrounding control-flow intact. Two complementary low-overhead mitigations are available.

The first is a loop (or recursion) counter assertion. At the end of each outer stage, assert that the inner loop counter reached its expected target ($j = m/2$, and ultimately $s = \log_2(n)$), aborting otherwise. The check sits outside the faulted loop and adds no asymptotic overhead to the $\Theta(n \log n)$ butterfly work. A fault model capable of simultaneously suppressing the loop body and advancing the counter to its terminal value is strictly stronger than the single-instruction skip assumed by our attack, and is substantially harder to realise in practice.

The second is an arithmetic checksum that exploits an algebraic identity of the transform itself. The forward NTT satisfies $\sum_j y_j = n \cdot a_0$, where a_0 is the constant coefficient of the input. After the batch transform, the prover sums the output vector, compares the result against $n \cdot a_0$, and aborts on mismatch. A corrupted leaf at position j changes the sum by $a_{\text{bitrev}(j)} - f(\omega^j)$, a random-looking field element, and each corrupted leaf is therefore detected with probability $1 - 1/|\mathbb{F}|$. The cost is $n - 1$ additions and a single multiplication, negligible against the $\Theta(n \log n)$ cost of the transform itself.

Table 14: DIT Cooley–Tukey loop-skip attack successes in **spike-fi**, across 30 experiments, varying n and p_f , with $o = 64$.

$n \setminus p_f$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
128	0	0	0	2	2	5	18	25	30	30	30
256	0	0	0	0	0	8	14	26	29	30	30
512	0	0	0	0	1	4	9	17	29	30	30
1024	0	0	0	0	0	1	7	18	29	30	30

Table 15: DIT Cooley–Tukey loop-skip attack successes in **spike-fi**, across 30 experiments, varying o and p_f , with $n = 512$.

$o \setminus p_f$	0.0	0.1	0.2	0.3	0.4	0.5	0.6	0.7	0.8	0.9	1.0
32	0	0	0	0	0	2	8	18	26	30	30
64	0	0	0	0	0	3	8	18	27	29	30
128	0	0	0	0	0	3	10	25	29	30	30
256	0	0	0	0	3	4	13	23	30	30	30

3.8 General Countermeasure Against Proof Tampering

In the non-interactive setting, the prover holds the entire proof locally before transmission and can therefore run the STARK verifier on its own output as a final sanity check. Indeed, faults that corrupt the committed witness or trace are caught with overwhelming probability by the checks involved in the FRI protocol. Since the verification cost is dominated by proving cost, and any inconsistency detected allows the prover to abort before the corrupted openings are sent, this mechanism adds only a small overhead. However, faults injected into the query answers or FRI layer leaves after the commitment can still go undetected.

We emphasize that this countermeasure does not apply to interactive provers, where openings to challenges have already been sent before finalizing the entire proof. Per-round consistency checks (e.g., recomputing a Merkle root before releasing it) remain possible, but cannot cover FRI queries that depend on later verifier challenges.

4 Conclusion

In this work, we have detailed simple fault injection attacks against zkSTARK provers capable of violating the ZK property, leaking private information to the verifier. These attacks make use of well understood fault models. This work constitutes the first exploration of fault injection attacks specifically tailored to the zkSTARK protocol. To the best of our knowledge, this is the first fault injection work to target general-purpose ZK proving systems. We attack a variety of primitives, which have varying degrees of use in real zkSTARK implementations, ranging from Horner polynomial evaluation (used only in interactive configurations) to the Cooley–Tukey NTT batch evaluation trick (used ubiquitously). Future work should explore the possibilities of more empowered adversaries capable of mounting more nuanced attacks.

References

- [0xP] 0xProject. OpenZKP. <https://github.com/0xProject/OpenZKP>. Git ref inspected via remote HEAD: c28a5c66b6ee.
- [Azt21] Aztec Protocol. Aztec: Fast Privacy with ZK² Rollup, 2021. Technical whitepaper.

- [BBHR18a] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity. In *ICALP, LIPIcs*, pages 14:1–14:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.
- [BBHR18b] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *IACR Cryptol. ePrint Arch.*, 2018:46, 2018.
- [BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *TCC (B2)*, Lecture Notes in Computer Science, pages 31–60, 2016.
- [BSCG⁺14] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474. IEEE, 2014.
- [CT65] James W. Cooley and John W. Tukey. An Algorithm for the Machine Calculation of Complex Fourier Series. *Mathematics of computation*, 19(90):297–301, 1965.
- [Eli] Eli Ben-Sasson et al. libSTARK. <https://github.com/elibensasson/libSTARK>. Git ref inspected: 11c72adf909e.
- [Fac] Facebook / Winterfell authors. Winterfell. <https://github.com/facebook/winterfell>. Git ref inspected: 2f78ee9bf667.
- [FS86] Amos Fiat and Adi Shamir. How to Prove Yourself: Practical Solutions to Identification and Signature Problems. In *CRYPTO*, Lecture Notes in Computer Science, pages 186–194. Springer, 1986.
- [GDD⁺26] Lior Goldberg, Maya Dotan, Ittay Dror, Gideon Kaempfer, Nir Levi, Noa Oved, Arad Reder, Anat Veredgorn, and Noa Wolfgor. Scalable Compliant Privacy on Starknet. *IACR Cryptol. ePrint Arch.*, 2026:474, 2026.
- [GS21] Jacob T. Grycel and Patrick Schaumont. SimpliFI: Hardware Simulation of Embedded Software Fault Attacks. *Cryptogr.*, 5(2):15, 2021.
- [Guia] Guild of Weavers. Distaff. <https://github.com/GuildOfWeavers/distaff>. Git ref inspected: fad92ce59292.
- [Guib] Guild of Weavers. genSTARK. <https://github.com/GuildOfWeavers/genSTARK>. Git ref inspected via remote HEAD: 6172f2bfb6a8.
- [HK24] Ulrich Haböck and Al Kindi. A note on adding zero-knowledge to STARKs. *IACR Cryptol. ePrint Arch.*, 2024:1037, 2024.
- [Hor19] William George Horner. A new method of solving numerical equations of all orders, by continuous approximation. *Philosophical Transactions of the Royal Society of London*, (109):308–335, 1819.
- [KSV13] Dusko Karaklajic, Jörn-Marc Schmidt, and Ingrid Verbauwhede. Hardware designer’s guide to fault attacks. *IEEE Trans. Very Large Scale Integr. Syst.*, 21(12):2295–2306, 2013.
- [Lam] LambdaClass. Lambdaworks. <https://github.com/lambdaclass/lambdaworks>. Git ref inspected: 3c8d8f65546c.

- [MAKK24] Puja Mondal, Supriya Adhikary, Suparna Kundu, and Angshuman Karmakar. ZKFault: Fault Attack Analysis on Zero-Knowledge Based Post-quantum Digital Signature Schemes. In *ASIACRYPT (8)*, Lecture Notes in Computer Science, pages 132–167. Springer, 2024.
- [Mata] Matter Labs. era-boojum. <https://github.com/matter-labs/era-boojum>. Git ref inspected: 5a8c4035a9e1.
- [Matb] Matter Labs. Hodor. <https://github.com/matter-labs/hodor>. Git ref inspected via remote HEAD: 76fc894f199c.
- [Mid] Miden. air-script. <https://github.com/0xMiden/air-script>. Git ref inspected via remote HEAD: 38d1394bb11a.
- [Mid26] Midnight Network. Proof Server, Feb 2026.
- [MRS25] Shibam Mukherjee, Christian Rechberger, and Markus Schafner. Cache Timing Leakages in Zero-Knowledge Protocols. In *AFT, LIPIcs*, pages 1:1–1:26. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2025.
- [Plo] Plonky3 authors. Plonky3. <https://github.com/Plonky3/Plonky3>. Git ref inspected via remote HEAD: 4aed8fe4195d.
- [Pola] Polygon. zkevm-prover. <https://github.com/0xPolygon/zkevm-prover>. Git ref inspected: 96e02add28c0.
- [Polb] Polygon Zero. Plonky2. <https://github.com/0xPolygonZero/plonky2>. Git ref inspected via remote HEAD: 5d9da5a65bbc.
- [RISa] RISC Zero. risc0. <https://github.com/risc0/risc0>. Git ref inspected via remote HEAD: e459039cc402.
- [risb] riscv-software-src contributors. Spike: RISC-V ISA Simulator. <https://github.com/riscv-software-src/riscv-isa-sim>. GitHub repository, accessed 15 July 2026.
- [RIS23] RISC Zero, Inc. RISC Zero zkVM, 2023. Zero-knowledge virtual machine for verifiable computation.
- [SCA18] Laurent Simon, David Chisnall, and Ross J. Anderson. What You Get is What You C: Controlling Side Effects in Mainstream C Compilers. In *EuroSP*, pages 1–15. IEEE, 2018.
- [SLP⁺25] Moritz Schneider, Daniele Lain, Ivan Puddu, Nicolas Dutly, and Srdjan Capkun. Breaking Bad: How Compilers Break Constant-Time Implementations. In *AsiaCCS*, pages 1690–1706. ACM, 2025.
- [Staa] StarkWare. ethSTARK. <https://github.com/starkware-libs/ethSTARK>. Git ref inspected via remote HEAD: 0eb683bf9dc9b.
- [Stab] StarkWare. Stone Prover. <https://github.com/starkware-libs/stone-prover>. Git ref inspected: 1414a545e4fb.
- [Stac] StarkWare. Stwo. <https://github.com/starkware-libs/stwo>. Git ref inspected: 5ea059737572.
- [Sta24] StarkWare Industries. StarkNet Documentation, 2024.

-
- [Suc] Succinct Labs. SP1. <https://github.com/succinctlabs/sp1>. Git ref inspected: 3a98ea24252f.
- [TBP20] Florian Tramèr, Dan Boneh, and Kenny Paterson. Remote Side-Channel Attacks on Anonymous Transactions. In *USENIX Security Symposium*, pages 2739–2756. USENIX Association, 2020.
- [Val] Valida. valida. <https://github.com/valida-xyz/valida>. Git ref inspected via remote HEAD: 5058de8573e2.
- [ZDCT13] Loïc Zussa, Jean-Max Dutertre, Jessy Clédière, and Assia Tria. Power supply glitch induced faults on FPGA: An in-depth analysis of the injection mechanism. In *IOLTS*, pages 110–115. IEEE, 2013.